Ricki Pierce
Semester Project Report
Image Processing – BE 7910
Due May 2nd, 2025

Introduction

Subject Matter: One of the primary challenges in O&P is the lack of detailed quantitative data to assess amputee progress. Currently, the most widely used assessment tool, the Amputee Mobility Predictor (AMP), categorizes patients into K-levels based on basic functional tasks like sit-to-stand movements[1,2]. However, AMP scores fail to capture the nuances of amputee mobility, as they lack the granularity to distinguish between those who struggle with a task and those who perform it easily. For example, in the sit-to-stand portion of the AMP test, patients are simply rated as either 0 (unable to stand) or 1 (able to stand), with no differentiation between varying levels of difficulty. This lack of detail limits our understanding of an amputee's true capabilities. Clinics typically rely on basic measurements, such as limb circumference and the distance a patient walks, with little quantitative tracking beyond that. Insurance companies depend on clinic notes and K-levels to determine authorization and coverage[3]. There is a need for more detailed and quantitative analysis of amputee mobility to better inform clinical decisions and improve patient outcomes.

Motion capture has been explored as a potential solution for bringing quantitative data into clinical settings. Traditional marker-based systems and mounted foot force plates can provide detailed movement analysis, but they are expensive, time-consuming, and require specialized training to set up and operate effectively. These systems are often fixed installations, limiting their use in most clinical environments. Even if clinics could afford them, clinicians and technicians often lack the time and expertise to use the systems properly.

A promising alternative to marker-based systems is markerless motion capture. OpenCap[4] uses OpenPose[5,6], a pose estimation algorithm from Carnegie Mellon, to track joint positions without the need for physical markers. While OpenCap has been used in sports medicine and rehabilitation, its application to amputees remains largely unexplored. The *only* known study on OpenCap with amputees was published in December 2024, but it utilized a small sample size of only five right transfemoral amputees[6]. The study attempted to assess various gait parameters, but it failed to explain how the amputee data was compared to normative data. It was unclear whether the normative data was sourced from healthy individuals using OpenCap or from a gold-standard method for amputees[7]. The Sit2Stand.ai algorithm aimed to automate sit-to-stand assessments. The osteoarthritis study recruited 493 subjects across the United States to capture videos of them performing the sit-to-stand task[8,9]. Low correlation values suggest that Sit2Stand.ai is not a valid tool for clinical use. Additionally, the study had several limitations, including a lack of standardization in the experimental setup, such as varying chair types and heights and foot and arm positions. Moreover, the validation studies were conducted on only 11 healthy individuals, which further undermines the generalizability of the results[8,9].

Despite the growing use of markerless motion capture systems, there remains a lack of validated normative models specific to the amputee population. A well-defined normative model for the amputee population would provide clinicians with essential benchmarks for evaluating patient outcomes, optimizing prosthetic designs, and improving rehabilitation protocols. Without such a model, it's challenging to distinguish between typical and atypical movement patterns, which can delay effective treatment.

Application: My master's research addresses a major gap in the existing literature by validating OpenCap's markerless motion capture system for use with amputee populations and introducing uncertainty quantification in motion capture and GRF data analysis. Only one study to date has attempted to use OpenCap with amputees, and it was limited in scope and rigor. My work will validate OpenCap's GRF and joint angle estimates against gold-standard data from foot force plates and a marker-based system, providing a more thorough and reliable assessment. Additionally, most motion capture studies report GRFs or joint angles without acknowledging the uncertainty in these measurements. By using Monte Carlo simulations, my research will produce uncertainty bounds that offer greater clarity. This research has the potential to improve clinical decision-making, enhance rehabilitation outcomes, and support the integration of data-driven practices into O&P workflows. The development of a validated, markerless motion capture system for amputee mobility will provide clinicians with objective and reliable data that can inform decision-making, help design more effective prosthetic devices, and improve patient outcomes. Furthermore, the introduction of uncertainty quantification will contribute to more accurate assessments of patient mobility, reducing ambiguity in clinical evaluations and insurance authorizations.

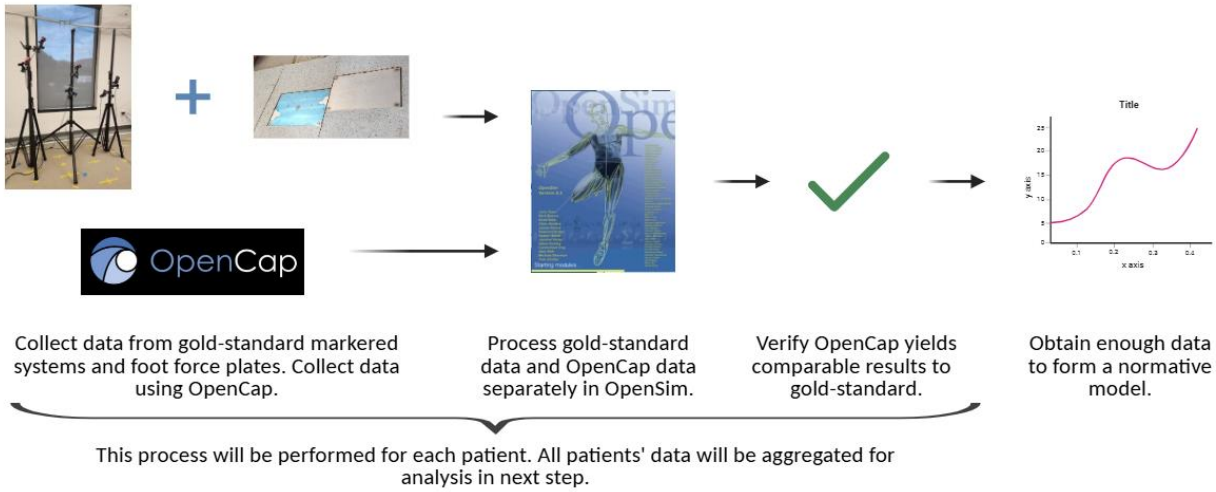

Figure 1. Proposed approach for Master's project.

For this semester's project for BE 7910, I developed a Python-based tool to automatically filter video data collected during movement tasks. Because this research involves capturing a large number of videos, manually reviewing each one would be inefficient. To streamline preprocessing, the program will analyze each video and calculate the percentage of time the subject remains in the frame using edge detection techniques. The script will generate a summary table indicating which videos were

processed and the proportion of usable frames in each, allowing for faster and more reliable selection of videos for further analysis.

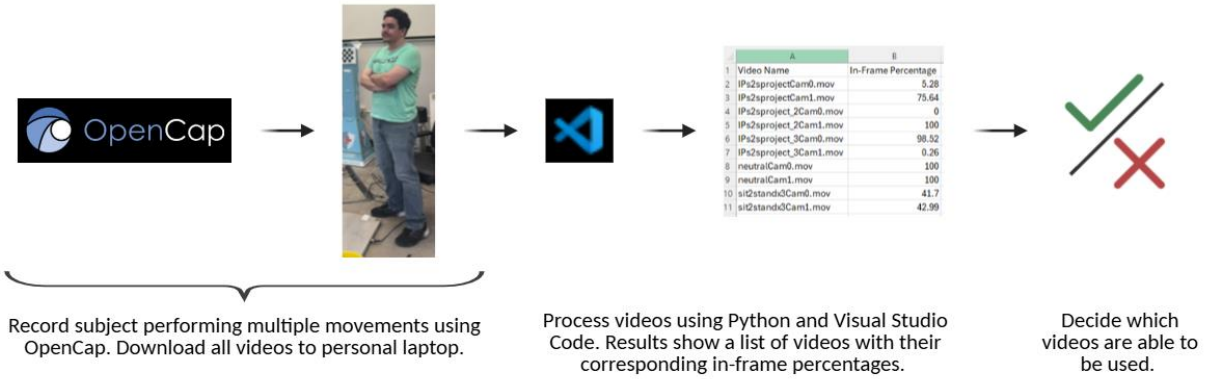

Figure 2. Proposed approach for BE 7910 project.

Image System

OpenCap Overview: OpenCap is an open-source markerless motion capture system designed to estimate 3D human body joint positions and orientations using Apple devices. It enables biomechanical analysis without the need for traditional motion capture labs, reflective markers, or expensive equipment. OpenCap requires synchronized video from at least two calibrated iOS device cameras, placed at different angles[4].

The system models each iOS camera using a 15-parameter pinhole camera model and determines intrinsic parameters (principal point, focal length, distortion coefficients, etc.) through a pre-computed database of recent iOS devices. These parameters are automatically loaded based on the specific device and recording settings. Calibration begins with a single image of a precision-manufactured 720x540 mm checkerboard placed in view of all cameras. From this, OpenCap computes each camera's extrinsic parameters, which define the position and orientation of the camera relative to a global reference frame[4].

After calibration, users can simultaneously record videos on both devices. Videos are captured at a resolution of 720×1280 pixels and a frame rate of 60 Hz, with a fixed camera focus distance. OpenCap uses two deep learning-based 2D pose detection algorithms: OpenPose[5,6] and HRNet[10]. Both output body keypoints for each video frame, along with confidence scores ranging from 0 to 1. OpenCap then applies custom algorithms to handle occluded keypoints and synchronize frames across videos[4].

Using the synchronized 2D data, OpenCap reconstructs 3D positions via triangulation using a Direct Linear Transformation algorithm. Each camera’s contribution is weighted according to the confidence score of the detected keypoints. OpenCap also employs two long short-term memory networks trained on 108 hours of synthetic motion capture data to predict the 3D positions of 43 anatomical markers from the 20 triangulated video keypoints[4].

Before movement recording, the user is guided to capture a neutral standing pose, which OpenCap uses to scale a 33-degree-of-freedom musculoskeletal model in OpenSim (according to Lai et al.[11] and Uhlrich et al.[12] models) to the subject. OpenCap then allows recording of movements. The resulting joint trajectories are low-pass filtered

using a fourth-order, zero-lag Butterworth filter with a 12 Hz cutoff for gait and 30 Hz for other movements[4].

Output includes visualizations, 3D joint trajectories, and CSV files. The overall reliability of OpenCap depends on precise camera calibration and placement, lighting and occlusion conditions, and the accuracy of the underlying pose detection algorithms[4].

Project Overview: For this project, video data is recorded using two iPhones, an iPhone 14 (Cam 0) and an older lab equipment model (Cam 1). Although the recordings are made using iPhones, the videos are not stored on the devices themselves. Instead, they are uploaded directly to the OpenCap system, where both video capture and processing are handled. Once the data is processed, videos are downloaded from OpenCap in .mov format to a personal computer for analysis.

In the biomechanics lab on campus, a controlled recording environment was set up to capture subjects performing movements on foot force plates. Two tripods were positioned approximately five feet away from the subject, at different angles, ensuring that the calibration QR code from OpenCap was roughly at eye level within the camera's field of view. Each tripod was equipped with one iPhone, and the cameras were calibrated using the calibration checkerboard.

Once the system is calibrated and synced, the subject was instructed to perform a movement, which was recorded and automatically uploaded to the OpenCap account. After the recording session, the videos were downloaded from OpenCap to a personal computer for further processing and analysis.

Objectives

1. Implement edge detection algorithms to analyze individual frames and identify the subject's position within the video.
2. Quantify subject visibility by calculating the percentage of frames in which the subject remains fully within the camera frame.
3. Automate batch video processing to efficiently analyze large datasets and minimize the need for manual video review.
4. Output results in a clear, usable format (e.g., summary tables) to support downstream analysis and decision-making.

Procedure

Overview: This Python code processes a folder of video files to perform "skeletonization", a form of human pose estimation, and evaluates whether a person remains fully in frame throughout the video. The code uses machine learning techniques via the MediaPipe Pose model, a human pose estimation algorithm developed by Google. Pose landmarks are identified and visualized for each frame, and results are logged into a CSV file***.*** Although the term "skeletonization" traditionally refers to morphological thinning or medial axis transformation[13] in image processing, here it's used more broadly to mean identifying a skeleton-like representation of the human body using landmark detection.

The code begins by importing several libraries necessary for video processing and analysis. OpenCV[14] is used for reading video frames, performing color conversions, and visualizing results. MediaPipe Pose[15] performs human pose estimation.

Pymediainfo[16] extracts metadata from video files such as codec, resolution, bit rate, etc. CSV[17], OS[18], Platform[19], and Subprocess[20] facilitate saving results, interacting with the operating system, and automating tasks such as opening files. Random[21] selects a random frame from the video for visualization.

MediaPipe Pose[15] uses a two-stage machine learning model to detect 33 anatomical landmarks on the human body. The convolutional neural network (CNN) uses models trained on large datasets (like BlazePose[22] and COCO[23]) to identify body landmark positions. These datasets are annotated with body markers, and training data consists of different camera angles and subjects. The first stage identifies the general region in the frame where a person is located. The models have been trained using standard object detection losses like cross-entropy and bounding box regression. The next stage of landmark estimation involves using another CNN to estimate the coordinates of each of the 33 key points within the region identified in the first stage. These include the nose, eyes, ears, mouth, shoulders, elbows, wrists, hips, knees, ankles, and toes. Each landmark comes with a set of coordinates and a visibility score. These landmarks are returned in results.pose_landmarks and drawn via mp_drawing.draw_landmarks(). mp_pose.POSE_CONNECTIONS defines which landmarks should be connected to draw the human skeleton.

Main Script: The script starts by defining the directory containing the video files using the Path library. The MediaPipe Pose[15] model is initialized with specified thresholds for detection and tracking confidence, both set to 0.9. This means the model will only report results when it is at least 90% confident in the detection. The MediaPipe Pose model is loaded using mp.solutions.pose, and the utility mp.solutions.drawing_utils is used for drawing landmarks and connections.

The function get_video_metadata(video_path) is used to open each video and extract key data including file size, video format, resolution, frames per second (FPS), frame count, duration (calculated as frame count divided by FPS), codec, bit rate, color space, and bit depth. This is done using both OpenCV for frame-related properties and pymediainfo for additional data.

The function analyze_subject_in_frame(video_path) opens each video and processes it frame by frame. Each frame is initially read in BGR format using OpenCV, but since MediaPipe Pose requires the frame to be in RGB format, a conversion is performed using cv2.cvtColor(frame, cv2.COLOR_BGR2RGB). The converted frame is passed to MediaPipe's pose model, and the detected landmarks (33 anatomical key points) are returned via results.pose landmarks. If landmarks are detected, the code calculates a bounding box around the subject using the minimum and maximum x and y values of the landmarks. (The normalized landmark coordinates are scaled to actual pixel dimensions.) A padding of 50 pixels is added to the bounding box to make sure the subject is completely in the outline.

The code keeps track of whether the subject remains fully in frame by comparing the bounding box coordinates to the video frame boundaries. If any part of the bounding box touches the edge of the frame or goes past it, that frame is considered out of frame. These out-of-frame intervals are logged, and the total number of frames where the subject is fully in frame is also tracked. After processing each video, the percentage of frames where the subject remains fully in frame is calculated and written to a CSV log file. The log includes the video name and the corresponding in-frame percentage.

A single random frame from the first video in the list is selected for visualization. The following stages are displayed using cv2.imshow(): original frame, frame with pose landmarks, frame with pose connections, and frame with landmarks and bounding box. After the visualizations, the code automatically opens the CSV log file for immediate review.

Alternative Script: The alternative script uses Canny edge detection. The cv2.createBackgroundSubtractorMOG2() method uses a statistical model, a Mixture of Gaussians (MoG), to model background pixels. This allows it to identify moving objects (foreground) by comparing each new frame against the learned background. The history=500 parameter specifies that the background model is built using the past 500 frames. The MOG2 algorithm fits a mixture of Gaussian distributions to model what is considered "normal" background. The detectShadows=True setting enables the algorithm to detect and exclude shadows from the foreground mask, improving motion segmentation. While the algorithm updates based on new frames, it does not involve making predictions based on learned parameters; it is simply a form of adaptive filtering using statistical parameters from past frames to adjust a baseline for detecting motion.

In this code, each video frame is first converted to grayscale to simplify motion detection. The background subtractor is then applied to extract the moving parts of the frame (the foreground). To highlight object boundaries, the Canny edge detection algorithm is used, which identifies areas with sharp intensity gradients. These edges are then thickened using morphological dilation with a 5×5 kernel, which helps bridge gaps and close small holes in the edges.

Contours are extracted from the dilated edge image to identify object shapes. To isolate the subject, the code filters contours based on area and aspect ratio and keeps only those located within a plausible region of the frame. Among the remaining candidates, the largest contour is assumed to represent the person. If a valid contour is detected, it is drawn in green. However, if the detected person is too close to the frame boundaries, the contour is drawn in red to indicate that the subject is partially out of frame. The code also logs the time intervals when the subject is out of the frame or near the edge.

Results

Figure 3 illustrates the progression of the pose estimation pipeline, beginning with the RGB frame, followed by landmark detection and connections, and finally, the drawing of a bounding box that encompasses the subject.

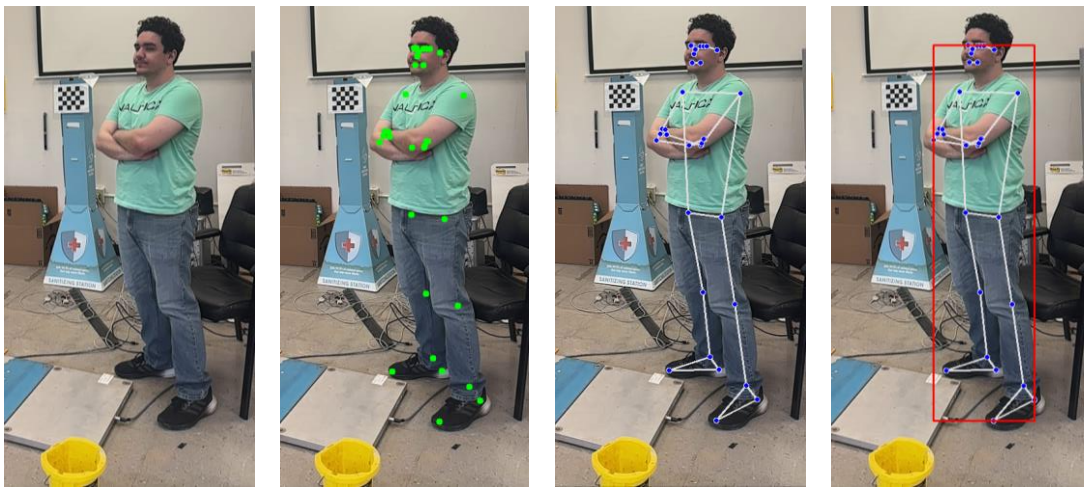

Figure 3. Panel showing original RGB image, landmark detection, landmarks connected, and bounding box from left to right. Notice this example does not include padding. Displayed frame #750 from the video IPs2sprojectCam1.mov.

Across the ten videos analyzed, the percentage of time the subject remained fully in frame ranged from 0% to 100%, with a mean of 56.44%. Videos 3 and 6 demonstrated the lowest in-frame percentages at 0% and 0.26%, respectively (Figure 4). These low values were expected, as the subject did not begin centered within the frame in either video. Videos 7 and 8 demonstrated the subject remaining in frame for 100% of the video (Figure 4). This was used as a verification case, since these neutral videos were used in the initial calibration of the OpenCap set up where the subject had to be completely visible.

	A	B
1	Video Name	In-Frame Percentage
2	IPs2sprojectCam0.mov	5.28
3	IPs2sprojectCam1.mov	75.64
4	IPs2sproject_2Cam0.mov	0
5	IPs2sproject_2Cam1.mov	100
6	IPs2sproject_3Cam0.mov	98.52
7	IPs2sproject_3Cam1.mov	0.26
8	neutralCam0.mov	100
9	neutralCam1.mov	100
10	sit2standx3Cam0.mov	41.7
11	sit2standx3Cam1.mov	42.99

Figure 4. Example .csv file opened in Excel showing list of videos and percentage of time subject was in frame.

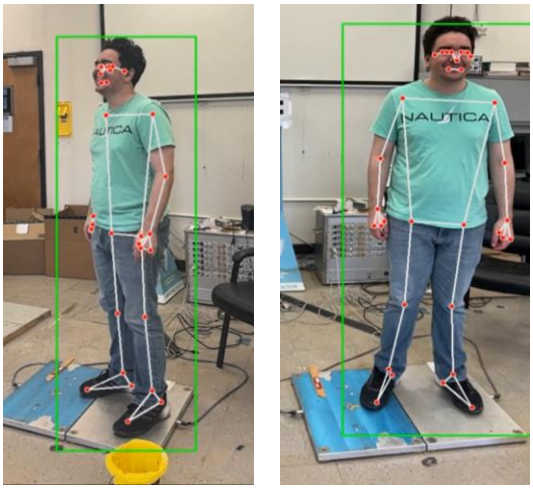

Figure 5. (Left) Landmark detection and bounding box when subject at angle vs (Right) subject facing camera.

Landmark detection generally aligned well with anatomical positions when the subject was oriented directly toward the camera. However, accuracy decreased when the subject appeared at an angle. In such cases, landmarks (especially those

corresponding to the hands and feet) occasionally jittered or were not detected. Figure 5 highlights this behavior: the left panel shows the subject rotated such that the right arm is partially obscured, yet few landmarks for the hands are detected. Similarly, Figure 3 captures the subject with arms crossed, hiding the hands, yet hand landmarks were still identified.

The right panel of Figure 5 shows the subject fully in frame. However, due to the hard-coded padding applied to the bounding box, the box itself extended beyond the image frame, resulting in the subject being incorrectly flagged as out of frame.

The code ran relatively slowly, primarily due to the computational demands of analyzing hundreds of frames per video. Processing time depended on both video length and resolution, with the visualization presenting the primary bottleneck. Detection consistency was also sensitive to the subject's motion speed and orientation; fast motions or angles led to more frequent landmark errors or omissions.

The results from the Canny edge detection aren't as clean or clear as those from the method using MediaPipe Pose. However, it can still yield practical and potentially more applicable results, depending on the research objectives. You might not need full, perfect edges. If you're just getting clusters of pixels that roughly outline the person, you can still use that to identify the topmost, leftmost, rightmost, or bottommost parts of the body. In that case, the entire contour is not needed. For example, the collection of points in the middle of Figure 6 seems to give a decent approximation of key points like the leftmost and rightmost edges. As long as the algorithm consistently captures those edge pixels, it has the potential to be used to define the edge of the person and even apply it to generate a better bounding box.

Figure 6. (Left) Foreground mask extracted from the background subtractor. (Middle) Canny edge detection algorithm identifies areas with sharp intensity gradients. The edges are thickened via dilation to bridge gaps and close small holes. Contours are extracted from the dilated edge image to identify object shapes. (Right) Original frame with bounding region applied.

Discussion

The primary outcomes were achieved. The developed codes successfully provide a rough but informative estimate of how consistently a subject remains fully in frame across a set of videos. This output is presented in a straightforward format, allowing for easy review and comparison. The goals of utilizing MediaPipe Pose for "skeletonization" and bounding box creation were met. This approach to edge detection quantified subject visibility by calculating the percentage of frames where the subject remained

entirely in frame. The script is flexible enough to process multiple videos in batch mode, as long as they are stored in the same folder, and its outputs are compatible with common analysis tools like Excel.

While successful, the implementation had its limitations. One notable limitation is that the bounding box is generated based on detected skeletal landmarks, rather than the subject's full visual outline. The decision to use MediaPipe Pose instead of traditional edge detection methods was driven by the need to mimic the behavior of current markerless motion capture methods, which rely on anatomical key point detection. The padding used to determine in-frame status was selected through trial and error and could be improved by dynamically adjusting based on subject size. As demonstrated in the right panel of Figure 5, this method introduces excessive empty space and can misclassify subjects who are technically visible but have bounding boxes extending outside the frame. A more refined approach might involve generating individual bounding boxes around key landmarks, producing a tighter fit around the subject. This would reduce the impact of padding and allow for more precise evaluation of subject visibility. Future work could explore a more flexible protocol, such as estimating the perimeter of the subject or calculating the distance between the outermost detected landmarks and the image boundaries. This approach would make in-frame determination relative to the person rather than the bounding box.

Several enhancements could improve both the speed and accuracy of this workflow. Processing time could be significantly reduced by removing visualization or by analyzing every *n*th frame or implementing parallel processing. Although visualization aids in debugging and interpretation, it slows down processing considerably. Additionally, using Canny edge detection with finer tuning and better thresholding could potentially yield more accurate boundary detection for defining whether a subject is in frame. Finally, the videos used in this study were downloaded from OpenCap. Acquiring raw video files directly from the source outside of OpenCap's system could reveal whether preprocessing by OpenCap affects detection accuracy. Another potential improvement would be the incorporation of smoothing across frames to reduce jitter in landmark detection, especially in frames with occlusion or fast motion.

Conclusion

This project successfully implemented Python-based pipelines to estimate human skeletal landmarks and quantify subject visibility in video recordings. By applying bounding boxes to landmark detections, the code estimated how often a subject remained fully in frame, producing both visual outputs and quantitative summaries in a CSV log. While processing speed and detection accuracy can be improved, particularly for non-frontal or occluded views, the results demonstrate the potential for automated video quality assessment in biomechanical applications. This offers a promising starting point for the preprocessing of large motion-video data sets.

References

1. Amputee Mobility Predictor. Physiopedia. 2014. https://www.physio-pedia.com/Amputee_Mobility_Predictor

2. Queensland. *Amputee Mobility Predictor Assessment Tool MASS SERVICE CENTRE DETAILS SUBMIT COMPLETED FORM TO MASS SERVICE CENTRE PRIVACY STATEMENT AND COLLECTION NOTICE*.; 2001. https://www.health.qld.gov.au/__data/assets/pdf_file/0040/939964/QALS-Clinic-AMPAT.pdf

3. CMS. Article - Lower Limb Prostheses - Policy Article (A52496). www.cms.gov. 2015. https://www.cms.gov/medicare-coverage-database/view/article.aspx?articleId=52496

4. Uhlrich SD, Falisse A, Łukasz Kidziński, et al. OpenCap: Human movement dynamics from smartphone videos. *PLOS Comput Biol*. 2023;19(10):e1011462-e1011462. doi:10.1371/journal.pcbi.1011462

5. Martinez G. *OpenPose: Whole-Body Pose Estimation*. Carnegie Mellon University; 2019. https://www.ri.cmu.edu/app/uploads/2019/05/MS_Thesis___Gines_Hidalgo___latest_compressed.pdf

6. Cao Z, Hidalgo Martinez G, Simon T, Wei SE, Sheikh YA. OpenPose: Realtime Multi-Person 2D Pose Estimation using Part Affinity Fields. *IEEE Trans Pattern Anal Mach Intell*. Published online 2019. doi:10.1109/tpami.2019.2929257

7. Leon SF. *Clinical Utility of OpenCap Motion Analysis in Lower Limb Prosthesis Users*. Undergraduate Honors Thesis. St. Mary's University; 2024. Accessed February 12, 2025. https://commons.stmarytx.edu/cgi/viewcontent.cgi?article=1054&context=honorstheses

8. Mobilize Center. Webinar: Sit2Stand – Assessing Health and Mobility from Smartphone Videos, Part 1 of 2. YouTube. 2023. https://www.youtube.com/watch?v=HGb9YCyNuBw

9. Mobilize Center. Webinar(Tutorial): Sit2Stand – Assessing Health and Mobility from Smartphone Videos, Part 2 of 2. YouTube. 2023. https://www.youtube.com/watch?v=Gej8VybGbHA

10. Sun K, Xiao B, Liu D, Wang J. Deep High-Resolution Representation Learning for Human Pose Estimation. IEEE Xplore. doi:10.1109/CVPR.2019.00584

11. Rajagopal A, Dembia CL, DeMers MS, Delp DD, Hicks JL, Delp SL. Full-Body Musculoskeletal Model for Muscle-Driven Simulation of Human Gait. *IEEE Trans Biomed Eng*. 2016;63(10):2068-2079. doi:10.1109/tbme.2016.2586891

12. Uhlrich SD, Jackson RW, Seth A, Kolesar JA, Delp SL. Muscle coordination retraining inspired by musculoskeletal simulations reduces knee contact force. *Sci Rep*. 2022;12(1):9842. doi:10.1038/s41598-022-13386-9

13. Burger W, Burge MJ. *Digital Image Processing*. Springer Nature; 2022.

14. Zelinsky A. Learning OpenCV---Computer Vision with the OpenCV Library (Bradski, G.R. et al.; 2008)[On the Shelf]. *IEEE Robot Autom Mag*. 2009;16(3):100-100. doi:10.1109/mra.2009.933612

15. Kim JW, Choi JY, Ha EJ, Choi J ho. Human Pose Estimation Using MediaPipe Pose and Optimization Method Based on a Humanoid Model. *Appl Sci*. 2023;13(4):2700-2700. doi:10.3390/app13042700

16. Introduction — pymediainfo 7.0.1 documentation. Readthedocs.io. 2025. Accessed January 1, 2025. https://pymediainfo.readthedocs.io/en/stable/introduction.html#getting-information-from-a-video

17. Python Software Foundation. csv — CSV File Reading and Writing — Python 3.8.1 documentation. Python.org. 2020. https://docs.python.org/3/library/csv.html

18. Python Software Foundation. os — Miscellaneous operating system interfaces — Python 3.8.0 documentation. Python.org. 2019. https://docs.python.org/3/library/os.html

19. Python Software Foundation. platform — Access to underlying platform's identifying data — Python 3.9.7 documentation. docs.python.org. https://docs.python.org/3/library/platform.html

20. subprocess — Subprocess management — Python 3.8.5 documentation. docs.python.org. https://docs.python.org/3/library/subprocess.html

21. Python. random — Generate pseudo-random numbers — Python 3.8.2 documentation. docs.python.org. https://docs.python.org/3/library/random.html

22. Bazarevsky V, Grishchenko I, Karthik Raveendran, Zhu T, Zhang F, Grundmann M. BlazePose: On-device Real-time Body Pose tracking. *ArXiv Cornell Univ*. Published online 2020. doi:10.48550/arxiv.2006.10204

23. Lin TY, Maire M, Belongie S, et al. Microsoft COCO: Common Objects in Context. *ArXiv Cornell Univ*. Published online 2014. doi:10.48550/arxiv.1405.0312